

```
In [89]: import matplotlib.pyplot as plt
import numpy as np
from sklearn import tree, ensemble
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import KFold
from collections import defaultdict
import pprint
```

```
In [90]: # Assignment Constants
RANDOM_STATE = 10
FIGSIZE = (12,8)
#### Use the following line before plt.plot(...) to increase the plot
# plt.figure(figsize=FIGSIZE)
```

Question 1

Use the breast cancer data set from Homework 0 to create a training set. Recall that the label is 0 if the patient's data indicates a malignant cancer and 1 otherwise. Compute the base rate of malignant cancer occurrence over the entire data set. In other words, what would be your best guess for the probability of malignant cancer of a single example using only the labels in the training set? This question is very simple, so try not to overthink it.

```
In [91]: cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

# label 0 = malignant
base_rate = np.sum(y == 0) / len(y)
print(base_rate)
```

0.37258347978910367

2

The goal is to build a decision tree that, based on the other features in the set, predicts whether or not a patient has malignant cancer. So this is a classification problem. Using `tree.DecisionTreeClassifier` and other functions in the scikit-learn library, one can build a decision tree and calculate both its training accuracy when fitted to the entire data set as well as its accuracy using 10-fold cross validation (which gives a better idea of true accuracy). In this question you will need to complete two sub-components:

(a)

(a) Make a plot visualizing the performance of a `tree.DecisionTreeClassifier` as you search for an optimal `max_depth` parameter. Vary the depth of your decision tree using `max_depth = 1, 2, . . . , 10` and record the results from the following evaluation procedures for each setting:

- The accuracy when training and testing on the full dataset.
- 10-fold cross-validated accuracy.

Plot the results of both evaluation procedures on the same plot with evaluation scores on the y-axis and max depth values on the x-axis. Use 10 as your random seed/state for the decision tree and the cross-validation. Use a legend to label both evaluation procedures.

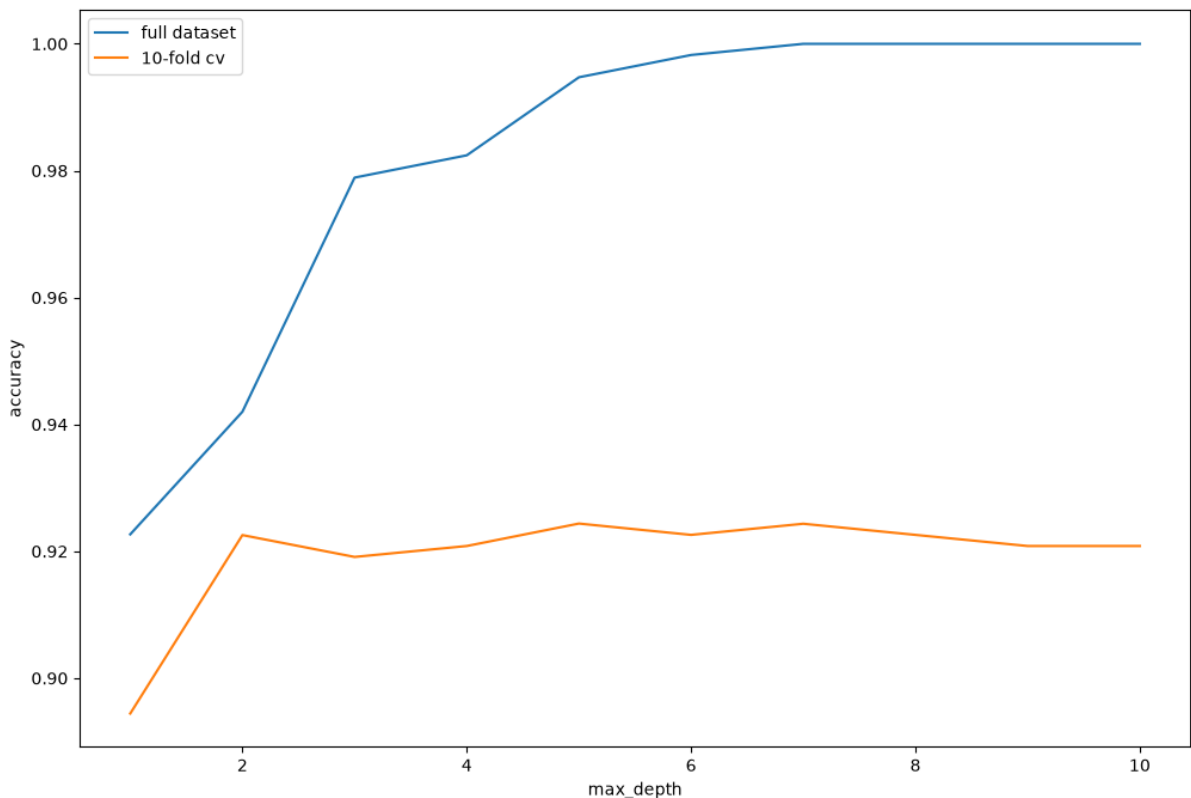
```
In [92]: kf = KFold(n_splits=10, shuffle=True, random_state=RANDOM_STATE)

depths = range(1, 11)
full_scores = []
cv_scores = []

for d in depths:
    clf = tree.DecisionTreeClassifier(max_depth=d, random_state=RANDOM_STATE)
    clf.fit(X, y)
    full_scores.append(clf.score(X, y))
    cv_scores.append(cross_val_score(clf, X, y, cv=kf).mean())

plt.figure(figsize=FIGSIZE)
plt.plot(list(depths), full_scores, label="full dataset")
plt.plot(list(depths), cv_scores, label="10-fold cv")
plt.xlabel("max_depth")
plt.ylabel("accuracy")
plt.legend()
plt.show()

print([float(s) for s in full_scores])
print([float(s) for s in cv_scores])
```



[0.9226713532513181, 0.9420035149384886, 0.9789103690685413, 0.9824253075571178, 0.9947275922671354, 0.9982425307557118, 1.0, 1.0, 1.0, 1.0]
 [0.8944235588972431, 0.9225563909774437, 0.919110275689223, 0.9208333333333334, 0.9243734335839597, 0.9225877192982456, 0.9243421052631579, 0.9225877192982456, 0.9208333333333332, 0.9208333333333332]

(b)

Answer the questions below based on the results of 2a. Write your answers in the corresponding field in the markdown cell that is present in the HW1 template notebook. Do this by double clicking the markdown cell and writing your answer directly in the cell. Pressing enter will re-render the markdown.

(i.)

What setting of `max_depth` gave the best accuracy w.r.t. the **full-dataset** accuracy? If more than one setting equaled the best accuracy, list each of the best settings.

Student answer here: 7, 8, 9, 10

(ii.)

What setting of `max_depth` gave the best accuracy w.r.t. the **cross- validated** accuracy? If more than one setting equaled the best accuracy, list each of the best settings.

Student answer here: 5

3.

This question explores random forest classifiers by using scikit-learn's `ensemble.RandomForestClassifier`. You will make two plots and answer questions about them.

(a)

For the first plot, use a `ensemble.RandomForestClassifier` and the best depth you found 2(b)ii as `max_depth`. We will now find the optimal setting of a second parameter, `n_estimators`. Vary the number of trees in the forest via the parameter `n_estimators` and plot its 10-fold cross-validated accuracy (use `n_estimators = 1, 2, \dots, 20`). Again, use 10 as your random seed for your classifier and cross-validation.

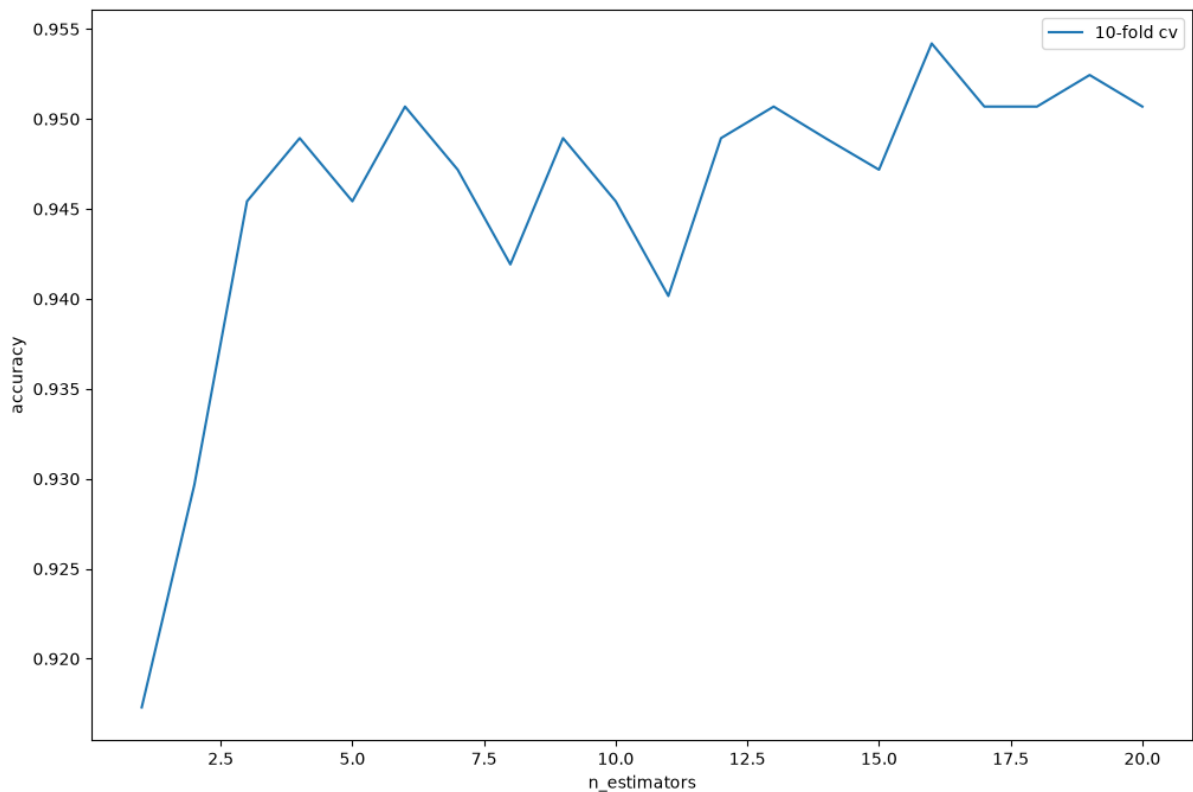
```
In [93]: BEST_DEPTH = 5 # from 2(b)ii

n_estimators_range = range(1, 21)
rf_n_scores = []

for n in n_estimators_range:
    rf = ensemble.RandomForestClassifier(
        n_estimators=n, max_depth=BEST_DEPTH, random_state=10
    )
    rf_n_scores.append(cross_val_score(rf, X, y, cv=kf).mean())

plt.figure(figsize=FIGSIZE)
plt.plot(list(n_estimators_range), rf_n_scores, label="10-fold cv")
plt.xlabel("n_estimators")
plt.ylabel("accuracy")
plt.legend()
plt.show()

print([float(s) for s in rf_n_scores])
```



[0.9172932330827068, 0.9296679197994988, 0.9454260651629072, 0.9489348370927317, 0.9454260651629072, 0.950689223057644, 0.9471804511278193, 0.9419172932330826, 0.9489348370927319, 0.9454260651629072, 0.9401629072681704, 0.9489348370927317, 0.950689223057644, 0.9489035087719297, 0.9471804511278196, 0.9541979949874687, 0.950689223057644, 0.950689223057644, 0.9524436090225563, 0.950689223057644]

(b)

Do you see an improvement using random forests versus using a single tree?
(Note: use the `n_estimators = 1` result as the result for a single tree.)

Student answer here: Yes, the random forest is clearly better than a single tree.
Every `n_estimators > 1` beats the single-tree result of 0.9173.

(c)

What setting of `n_estimators` gave the best accuracy w.r.t. the cross-validated accuracy?

Student answer here: `n_estimators = 16`

(d)

For the second plot, again use a `ensemble.RandomForestClassifier`, but this time you will fix the `n_estimators` parameter and again attempt to find the

optimal setting of a `max_depth`. Use your answer to 3c as the setting for `n_estimators` and follow the procedure from 2a to find the best setting for max depth. This time, only plot the results from cross-validation and not the full set, but the plot should be the same structure as in 2a otherwise (use `max_depth = 1,2,...,10`). Again, use 10 as your random seed.

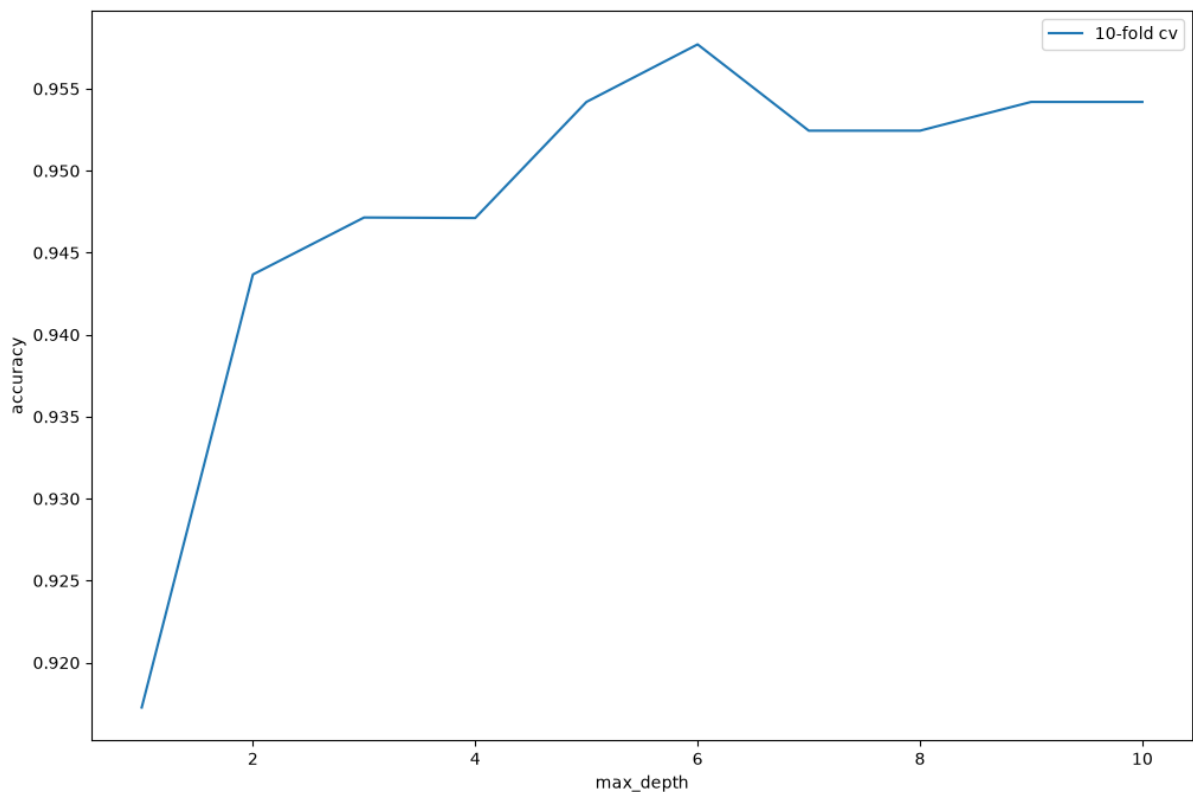
```
In [94]: BEST_N_ESTIMATORS = 16 # from 3(c)

rf_depth_scores = []

for d in depths:
    rf = ensemble.RandomForestClassifier(
        n_estimators=BEST_N_ESTIMATORS, max_depth=d, random_state=RAND
    )
    rf_depth_scores.append(cross_val_score(rf, X, y, cv=kf).mean())

plt.figure(figsize=FIGSIZE)
plt.plot(list(depths), rf_depth_scores, label="10-fold cv")
plt.xlabel("max_depth")
plt.ylabel("accuracy")
plt.legend()
plt.show()

print([float(s) for s in rf_depth_scores])
```



```
[0.9172619047619046, 0.9436716791979951, 0.9471491228070177, 0.9471177944862156, 0.9541979949874687, 0.9577067669172932, 0.9524436090225563, 0.9524436090225563, 0.9541979949874685, 0.9541979949874685]
```

(e)

In the plot in 3d, is the optimal setting of `max_depth` the same as in 2(b)ii? If not, what is the new optimal setting of `max_depth` ?

Student answer here: No, it is not the same as 2(b)ii. New optimal `max_depth` is 6.

4.

For this last question, we will explore the dependability of our estimates.

(a)

Make a plot using the following procedure:

i.

Using random state values from 0, 1, $\dots$, 99 calculate the 10-fold cross-validation accuracy of different `tree.DecisionTreeClassifiers` with max depth settings from 1, 2, $\dots$, 10. As before, you should use the same random state value for your classifier and cross-validation.

ii.

Then record the best max depth settings for each random state. Be sure to check whether multiple settings achieve the best accuracy.

Plot the counts for the best max depth settings as a bar chart with the max depth settings on the x-axis and the 'best parameter counts' on the y-axis (number of times that parameter was selected as the best max depth setting).

Note: this calculation might take some time. For debugging, try a smaller range of random states.

```
In [95]: counts = defaultdict(int)

for seed in range(100):
    kf_seed = KFold(n_splits=10, shuffle=True, random_state=seed)
    seed_scores = []
    for d in depths:
        clf = tree.DecisionTreeClassifier(max_depth=d, random_state=seed)
        seed_scores.append(cross_val_score(clf, X, y, cv=kf_seed).mean)
    best = max(seed_scores)
    for d, s in zip(depths, seed_scores):
        if s == best: # count every depth that ties
```

```

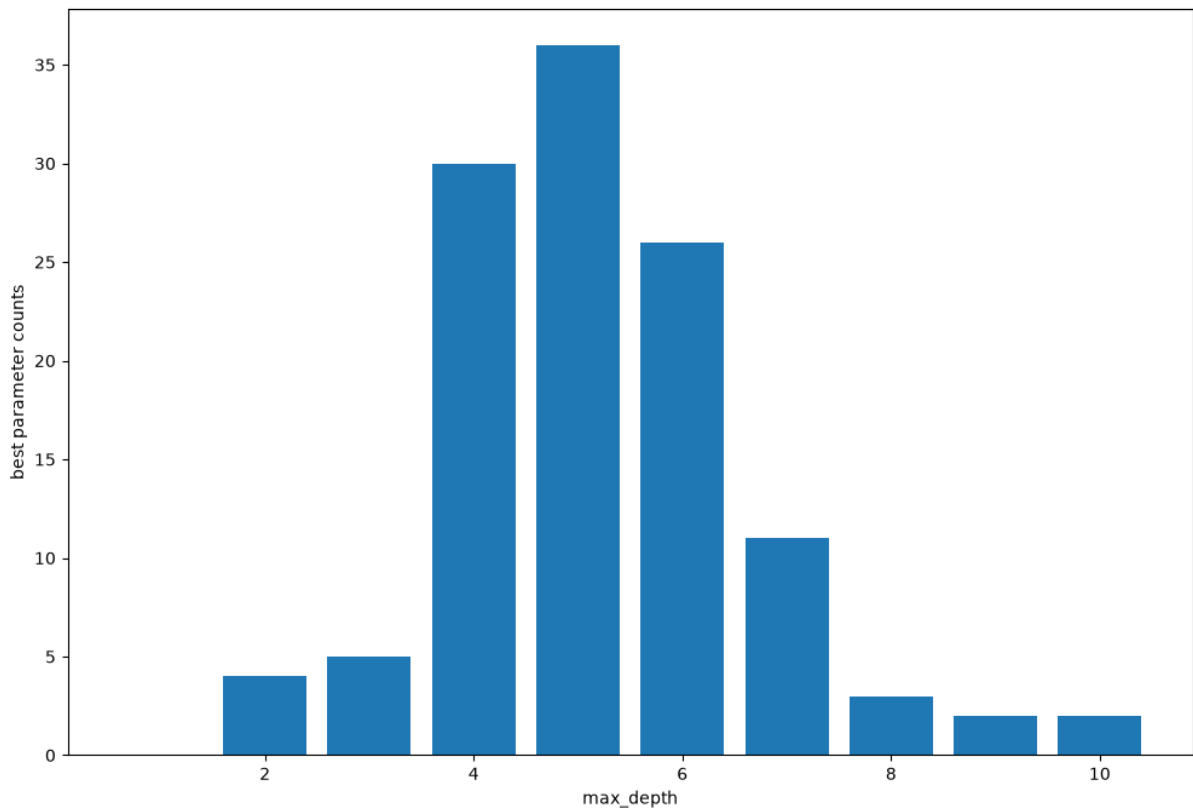
counts[d] += 1

best_counts = [counts[d] for d in depths]

plt.figure(figsize=FIGSIZE)
plt.bar(list(depths), best_counts)
plt.xlabel("max_depth")
plt.ylabel("best parameter counts")
plt.show()

print(dict(sorted(counts.items())))

```



{1: 0, 2: 4, 3: 5, 4: 30, 5: 36, 6: 26, 7: 11, 8: 3, 9: 2, 10: 2}

(b)

What are the top two most frequent parameter settings?

Student answer here: The two most frequent parameter settings were max_depth = 5 and max_depth = 4.